

A STUDY SERIES HANDBOOK



MACHINE LEARNING

VOLUME II

Model Types, Challenges, and Business Applications

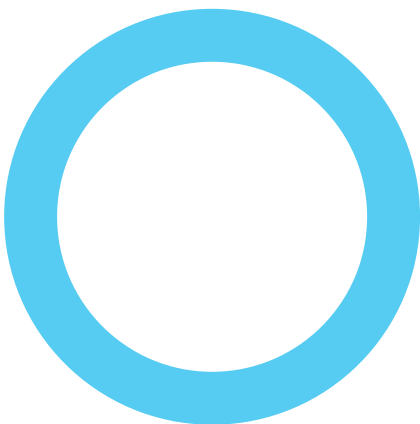
Batch vs. Online Learning ■ Instance- vs. Model-Based Learning
Ten Challenges in ML ■ B2B Applications

A Continuation of Volume I

From production behaviour to business applications

Adar's Study Series

Author: Adar Dey



How to Read This Volume

Volume I established what machine learning is and classified algorithms by the *amount of supervision* they receive. This companion volume asks a different set of questions: *how* a model is trained and deployed (batch vs. online learning), *how* it acquires knowledge (instance-based vs. model-based learning), *what can go wrong* in real projects (ten major challenges), and *where the technology earns its keep* (business-to-business applications).

The visual conventions are identical to Volume I: navy-headed boxes contain **core definitions**, teal-headed boxes contain **worked examples and case studies**, amber boxes highlight **one-line intuitions**, and every chapter closes with a *Key Takeaways* box for rapid revision.

Contents

How to Read This Volume	2
1 Types of Machine Learning Based on Training and Production Behaviour	5
1.1 What Is a Production Environment?	5
1.2 Batch Learning (Offline Learning)	5
1.2.1 How the Training Process Works	5
1.2.2 The Main Limitation: The Static Model Problem	6
1.2.3 The Re-Training Cycle	6
1.2.4 Limitations of Batch Learning	6
1.3 Online Learning	7
1.3.1 How It Works	8
1.3.2 Strengths and Risks	8
2 Instance-Based versus Model-Based Learning	9
2.1 Two Ways of Learning	9
2.2 Instance-Based Learning	9
2.2.1 Working Example	9
2.3 Model-Based Learning	10
2.3.1 Working Example	10
2.3.2 Parameters and Algorithms	10
2.4 Key Differences	12
3 Ten Major Challenges in Machine Learning	13
3.1 Data Collection	13
3.2 Insufficient and Unlabeled Data	13
3.3 Non-Representative Data	13
3.4 Poor Data Quality	14
3.5 Irrelevant Features	14
3.6 Overfitting	14
3.7 Underfitting	15
3.8 Software Integration	15
3.9 Offline Learning and Deployment	15
3.10 Hidden Operational Costs	15
4 Real-World Applications of Machine Learning: A B2B Focus	17
4.1 Beyond Consumer Products	17
4.2 Retail	18
4.2.1 Stock Forecasting	18
4.2.2 Customer Profiling and Targeted Marketing	18
4.2.3 Product Placement (Association Rule Mining)	18
4.3 Banking and Finance	18

4.3.1	Loan Sanctioning	18
4.4	Transportation and Logistics	18
4.4.1	Surge Pricing	18
4.4.2	Logistics and Route Optimization	19
4.5	Manufacturing	19
4.6	Social Media: The Twitter Example	19
4.6.1	Sentiment Analysis	19
4.6.2	Turning Data Into Profit	19

Chapter 1

Types of Machine Learning Based on Training and Production Behaviour

Volume I categorized machine learning algorithms by the **amount of supervision** involved — supervised, unsupervised, semi-supervised, and reinforcement learning. This chapter introduces a second, orthogonal axis of classification: **how the model is trained**, and **how it behaves once it is running in a production environment**.

1.1 What Is a Production Environment?

◇ Definition — Production Environment

The **production environment** is the live server on which a finished software system actually runs. When a machine learning application is built, it is not kept running on the developer's personal machine; it is **deployed** to a server. Users then interact with that server — via its IP address or an API — to obtain predictions. This live, running environment is called production.

Based on how a model behaves on the server, there are two broad types of learning process:

1. **Batch machine learning** (also called *offline learning*);
2. **Online machine learning**.

1.2 Batch Learning (Offline Learning)

Batch learning is the traditional, conventional way of training a machine learning model.

◇ Definition — Batch Learning

In **batch learning**, the model is trained on the *entire available dataset* in a single pass. It does not learn incrementally: once training is complete and the model is deployed, its parameters remain *fixed* until the next full re-training.

1.2.1 How the Training Process Works

1. **Uses the entire dataset.** The model is trained using the whole dataset at once.
2. **No incremental training.** Data is not split into small chunks or fed incrementally — the model is trained on the full batch of data in one go.

3. **Trained offline.** For large real-world datasets, loading and training directly on the production server would be costly and time-consuming, so data scientists train the model offline, typically on their own machines.
4. **Deployment.** Once the model is trained and its performance tested, the trained model is deployed to the production server.

Example — A Netflix Recommendation Engine

A recommendation engine built with batch learning is first trained completely offline on a data scientist's machine. Once ready, it is deployed to Netflix's servers, from where it starts recommending movies to users.

1.2.2 The Main Limitation: The Static Model Problem

Once a batch-learning model finishes training, it becomes **static** — it keeps working based on old information. But real business scenarios and data keep evolving over time.

For example, if a spam classifier is trained on today's data but spammers develop new tricks over the following year, the old static model will fail to catch the new spam and become obsolete. To stay useful, the model must be periodically updated with new data.

A batch model is a snapshot of the past: it keeps answering today's questions with yesterday's knowledge.

1.2.3 The Re-Training Cycle

To keep a batch model relevant, data scientists periodically re-train it in a repeating cycle (Figure 1.1):

1. collect new data continuously;
2. merge the new data with the old dataset;
3. re-train the model from scratch;
4. test the re-trained model;
5. deploy the updated model to the server.

This entire cycle repeats at a fixed interval — every 24 hours, every week, every month, or even every six months — depending on how quickly updates are needed and how large the dataset is.

1.2.4 Limitations of Batch Learning

Although batch learning is used in most situations, it fails or becomes problematic in three specific scenarios.

(a) Data Size Limitation

If data volume grows extremely fast and accumulates massively (e.g., social network data), batch learning becomes a bottleneck. Since the entire dataset must be loaded at once for training, the training tools may be unable to process such huge volumes of data.

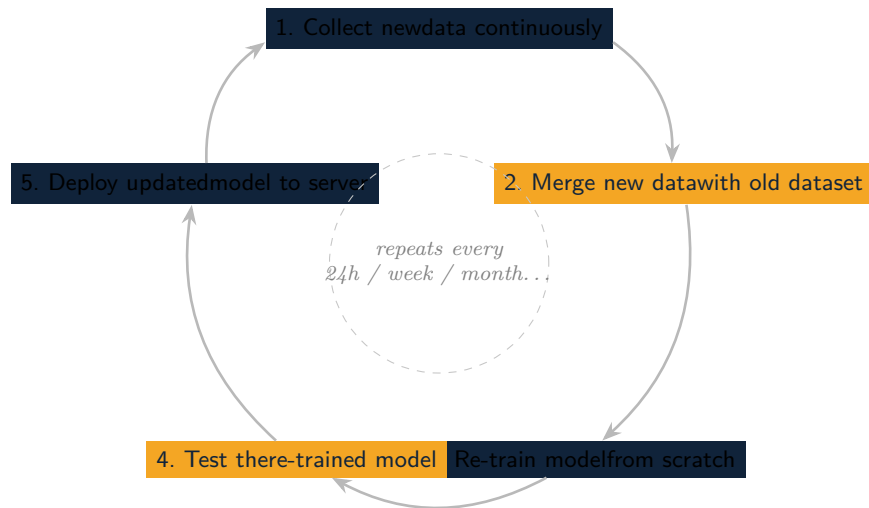


Figure 1.1: The batch-learning re-training cycle: the model is periodically rebuilt from scratch and redeployed.

(b) Hardware and Connectivity Limitation

If the model must run in an environment where frequent, instant connectivity with the server is impossible, updating the model becomes impossible.

Example — Satellite and Field Deployments

Consider a machine learning application built for military use in an extreme environment (such as Ladakh), or one deployed on a satellite where internet connectivity is limited or unavailable. Once deployed, the model cannot be retrieved, retrained with new data, and redeployed until it returns to a connected environment. Batch learning fails in such situations.

(c) Latency and Stale Response

Batch-learning models are typically updated only after 24 hours or more.

Example — Trending Stories on a Social Network

Imagine a model that curates trending stories on a social networking site and is updated once every 24 hours. Suddenly, breaking news goes viral: users become instantly interested, but a batch system cannot process this new interest for up to 24 hours. By the time the system updates, the news may have already lost relevance and the user's feed is flooded with stale stories. Batch learning therefore often fails to deliver time-critical information.

1.3 Online Learning

These limitations are exactly why **online learning** becomes necessary.

◇ Definition — Online Learning

In **online learning**, the model is trained *incrementally* by feeding it small groups of data (mini-batches), or even single data points, as they arrive. Each arrival triggers a small parameter update. The technique also enables *out-of-core* training: datasets so large that

they cannot fit in memory can simply be streamed through the model.

1.3.1 How It Works

Whenever a new mini-batch B_t arrives, the model parameters θ are nudged in the direction that best fits that batch:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t; B_t), \quad (1.1)$$

where $\mathcal{L}$ is the loss function and η is the **learning rate** — the hyperparameter that controls how fast the model adapts. A high η makes the system react quickly to new data but forget old data rapidly; a low η gives the system more inertia and slower adaptation.

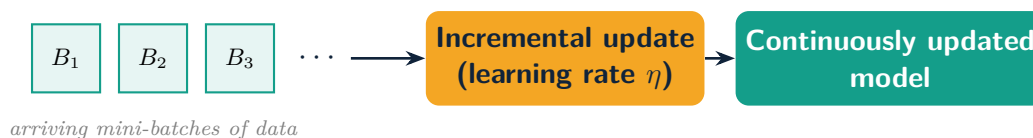


Figure 1.2: Online learning: the model is updated incrementally as small batches of data arrive; the learning rate η controls how strongly each batch influences the model.

1.3.2 Strengths and Risks

Strengths:

- updates are fast and cheap — a single data point can update the model in milliseconds;
- the system adapts to continuously drifting data (e.g., stock prices, trending content);
- arbitrarily large datasets can be trained on out-of-core, one mini-batch at a time.

Risks: an online system is sensitive to bad data. A single corrupted input can immediately degrade the model, so production systems require close monitoring and the ability to roll back. Online learning is also comparatively harder to implement than batch learning.

Related Tools

Dedicated libraries for online and incremental learning include vowpalwabbit.org and riverml.xyz.

★ Key Takeaways

- ✓ ML has a second classification axis: by training and production behaviour (in addition to the amount of supervision).
- ✓ Production is the live server environment where a deployed model serves real users.
- ✓ Batch learning trains once on the full dataset, deploys a static model, and relies on a periodic re-training cycle to stay relevant.
- ✓ Batch learning fails when data is massive, connectivity is absent, or latency matters.
- ✓ Online learning updates the model incrementally on arriving mini-batches; it adapts fast but must be monitored against bad data.

Chapter 2

Instance-Based versus Model-Based Learning

This classification is based on **how a model acquires knowledge**. It reveals how an algorithm actually interacts with data at prediction time — a question every practitioner should ask before trusting any model.

2.1 Two Ways of Learning

Machine learning models mainly learn in two ways — remarkably similar to how humans learn:

1. **Memorizing:** simply remembering the information.
2. **Generalizing:** trying to understand an underlying principle or concept.

Based on these two approaches, machine learning algorithms fall into two main categories: *instance-based learning* and *model-based learning*.

2.2 Instance-Based Learning

◇ Definition — Instance-Based Learning

Instance-based learning is the approach in which the algorithm literally holds onto — or memorizes — the training data points. All generalization is deferred to prediction time: given a new query, the algorithm compares it to the stored instances using a similarity or distance measure and answers from the most similar stored examples.

2.2.1 Working Example

Suppose we have data on students — their IQ, CGPA, and whether they were placed (Yes/No). This is a classification problem. An instance-based algorithm handles it as follows:

1. **Store the data.** It stores the training data exactly as it is.
2. **Scoring / prediction.** When a new student's data arrives (the *scoring instance*) and we ask whether they will be placed, the algorithm uses *no model and no rule*.
3. **Relies on similarity.** It depends purely on similarity or distance: it computes the distance between the new point and every other point in the dataset.
4. **Neighbor-based decision.** It considers a fixed number of nearest neighbors (e.g., 3 or 5). If most of these nearest neighbors were placed, the algorithm predicts that the new student will also be placed.

Core idea: “Where you stand tells us something about you.”

◇ Key Feature — Lazy Learning

The algorithm does *nothing* until a new query point arrives: the training/learning process is essentially postponed. For this reason, instance-based learning is also called **lazy learning** — it simply holds onto the data points. The classic example is **K-Nearest Neighbors (KNN)**.

2.3 Model-Based Learning

◇ Definition — Model-Based Learning

Model-based learning is the approach in which the algorithm tries to *discover the underlying principle or pattern* in the data. During training it infers a mathematical model of the input–output relationship; predictions are then made by evaluating that model — never by consulting the training data again.

2.3.1 Working Example

Using the same placement problem (IQ, CGPA, Placement), a model-based algorithm solves it differently:

1. **Mathematical relationship.** Given the data points, it derives a mathematical relationship (a function) between the inputs and the output.
2. **Decision boundary creation.** This relationship yields a decision boundary — a curve or equation — that separates the two classes (placed / not placed).
3. **Generalization rule.** The boundary *is* the model’s generalization rule: it captures the pattern rather than memorizing the data points.
4. **Prediction.** Once this decision function is built, predicting for a new student requires no training data at all. If the new point falls on one side of the boundary, the model predicts “placed”; on the other side, “not placed.”

2.3.2 Parameters and Algorithms

The goal of model-based learning is to build a model defined by a set of **parameters**. For example, a linear model

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \quad (2.1)$$

is completely described by its parameters $\theta_0, \theta_1, \dots, \theta_n$ — once they are learned, the training data can be discarded. A neural network, likewise, is defined by its learned weights.

Model-based algorithms include **linear regression**, **logistic regression**, **decision trees**, and most **neural network** models.

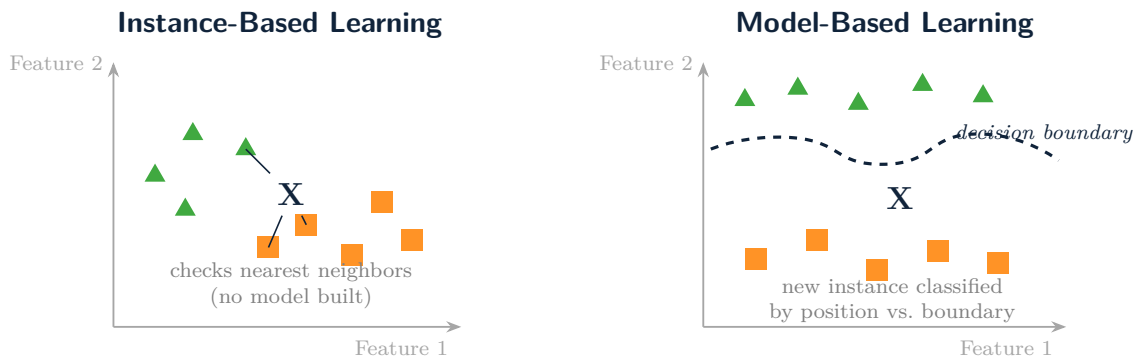


Figure 2.1: Instance-based learning (relies on nearest neighbors) versus model-based learning (relies on a learned decision boundary).

Table 2.1: Instance-based versus model-based learning.

Feature	Model-Based Learning	Instance-Based Learning
Model building	A parametric model (a mathematical equation) is built during training.	No model is built; only the data is stored.
Timing of learning	The generalization rule is found before scoring; training is completed upfront.	Learning/prediction is postponed; it only acts once a new point arrives.
Data requirement	Once the model is built, the training data can be discarded — the model alone is enough.	All training data must always be retained for prediction.
Generalization	A generalization rule is applied via the model.	No general rule; decisions are made by directly comparing to the incoming point.
Storage requirement	Low — only model parameters (e.g., an equation) are stored.	High — the entire dataset must be kept.
Also known as	—	Lazy Learning

2.4 Key Differences

★ Key Takeaways

- ✓ Models learn by memorizing (instance-based) or generalizing (model-based) — mirroring two modes of human learning.
- ✓ Instance-based algorithms store the data and compare by distance (KNN is the classic example); they are called *lazy* because no work happens until a query arrives.
- ✓ Model-based algorithms learn a parametric function or decision boundary; after training, the data can be discarded.
- ✓ Instance-based methods are storage-heavy; model-based methods are storage-light.
- ✓ Habit worth building: whenever you learn a new algorithm (e.g., logistic regression or KNN), first identify whether it is model-based or instance-based — this tells you how it actually makes decisions.

Chapter 3

Ten Major Challenges in Machine Learning

Building a real-world machine learning system involves far more than choosing an algorithm. Below are the ten challenges that data scientists and machine learning engineers encounter most often, from data acquisition to production cost.

3.1 Data Collection

Machine learning depends entirely on learning from data. For small projects, data is easily available from CSV files or platforms such as Kaggle. But in real company projects, collecting or aggregating data is a genuinely difficult task — often requiring APIs or web scraping, each with its own complications.

3.2 Insufficient and Unlabeled Data

- **Insufficient data.** More data generally improves model performance — even with a simple algorithm. This phenomenon was memorably titled *The Unreasonable Effectiveness of Data* (Halevy, Norvig, and Pereira, 2009). But not everyone has access to huge datasets, so performance can suffer when data is limited.
- **Lack of labeled data.** While raw data (like images) can be collected easily via APIs or scraping, correctly labeling each data point — e.g., marking which images are cats versus dogs — is tedious and time-consuming.

3.3 Non-Representative Data

If the training data does not accurately capture the full picture of a problem, the model's decisions can be wrong. Two distinct failure modes must be distinguished:

◇ Sampling Noise vs. Sampling Bias

Sampling noise: when the sample is *too small*, pure chance can make it unrepresentative — even if the sampling method is perfectly random.

Sampling bias: when the sampling *method* is not random, the sample can be unrepresentative *no matter how large it is*. A flawed method that systematically favors one group produces bias even in millions of records.

Example — Predicting the Cricket World Cup

Suppose you want to know who will win the Cricket World Cup, but you survey only people in India. The answer will likely be biased toward India — a *sampling bias*, because the sampling method excluded everyone else. Every group deserves a fair chance of representation for the results to be accurate.

3.4 Poor Data Quality

Real data contains many errors: junk values, missing values, outliers, and inconsistent formats. Data scientists reportedly spend around **60% of their total project time** — roughly eight months of a one-year project — simply cleaning and improving data quality. If data quality is poor, the algorithm cannot produce good results, because it depends entirely on the data it is given.

3.5 Irrelevant Features

Columns that contribute nothing to the prediction are called **irrelevant features**. As the saying goes: “*Garbage in, garbage out*” — feeding unnecessary features produces poor output.

Example — Marathon Participation

When predicting whether someone will participate in a marathon, age, weight, and height are relevant, but “location” is usually irrelevant and should be dropped. Sometimes two features are better combined into a new, more informative one — e.g., weight and height combined into body mass index, $BMI = w/h^2$. This process of designing, dropping, and combining features is called **feature engineering**.

3.6 Overfitting

◇ Definition — Overfitting

Overfitting occurs when a model memorizes the training data — including its noise — instead of grasping the underlying pattern. The model follows every single training point too closely, producing an overly complex function. Training performance looks excellent, but performance drops sharply on new, unseen data.

In terms of error, the three regimes are:

Underfitting:	E_{train} large,	E_{new} large,
Good fit:	E_{train} small,	E_{new} small,
Overfitting:	E_{train} very small,	E_{new} large,

where E_{train} is the error on training data and E_{new} is the error on unseen data. The standard antidotes to overfitting are simpler models, more training data, better features, and regularization (penalizing model complexity).

3.7 Underfitting

◇ Definition — Underfitting

Underfitting is the opposite problem: the model is *too simple* to capture the underlying structure of the data, and therefore performs poorly on both the training data and new data.

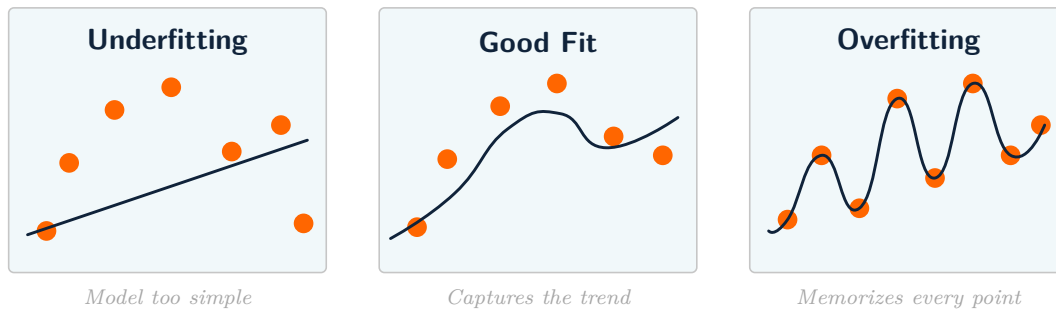


Figure 3.1: Underfitting (too simple to capture the trend), a good fit, and overfitting (a function that memorizes every training point), shown as three side-by-side panels.

3.8 Software Integration

The final goal of a machine learning project is to deliver it as part of a real software product — a recommendation system, a prediction service, an application feature. Integrating ML models across different platforms (Windows, Android, Linux, servers) is difficult. Because ML is still a relatively young technology, many software platforms do not yet support it stably (e.g., Java or older JavaScript). Ensuring the model works correctly on every platform is a major engineering challenge.

3.9 Offline Learning and Deployment

As Chapter 1 showed, in **offline learning** the model is trained once and uploaded to a server without automatic updates; updating it requires taking it offline, retraining with new data, and re-uploading. The alternative — continuously updating the model — is **online learning**, which is comparatively harder to implement.

Deployment — taking a model to production — is itself very difficult, even with top providers such as AWS. Real-time monitoring and production operations show that ML deployment still needs considerably more maturity than traditional software deployment.

3.10 Hidden Operational Costs

When deploying an ML model at large scale (e.g., for 100,000 users), significant hidden costs can appear unexpectedly. Because server-side ML optimization is not yet fully mature, controlling these costs is a major challenge (see, e.g., the literature on the cost of AI systems).

◇ MLOps — Machine Learning Operations

To tackle all of these challenges, a new field has emerged: **MLOps**. It focuses on deploying ML products to servers and managing their operational lifecycle and cost — and is currently one of the fastest-growing fields in the industry.

★ Key Takeaways

- ✓ Real-world ML is dominated by data problems: collection, quantity, labeling, representativeness, and quality.
- ✓ Sampling noise comes from samples that are too small; sampling bias comes from sampling methods that are not random — and bias persists even in huge samples.
- ✓ Around 60% of project time is typically spent on data cleaning: garbage in, garbage out.
- ✓ Overfitting memorizes the training data (poor generalization); underfitting is too simple even for the training data.
- ✓ Deployment, integration, and hidden costs are the last-mile problems — which is why MLOps has emerged as a discipline of its own.

Chapter 4

Real-World Applications of Machine Learning: A B2B Focus

Machine learning is not a technology of the future — it is already part of our everyday lives. Many software products have ML built in, such as Facebook’s friend recommendations, or product and video recommendations on YouTube and Amazon.

4.1 Beyond Consumer Products

Students often think of ML usage only in terms of consumer products (**B2C**) like WhatsApp or YouTube. However, ML plays an equally — if not more — important role in **Business-to-Business (B2B)** use cases, helping companies run their operations and generate profit. In fact, companies often earn far more money from B2B applications of ML than from B2C applications.

Five major sectors illustrate this well: **Retail**, **Transportation**, **Manufacturing**, **Banking**, and **Social Media**. ML is also widely used in space exploration, healthcare, legal, and defense.

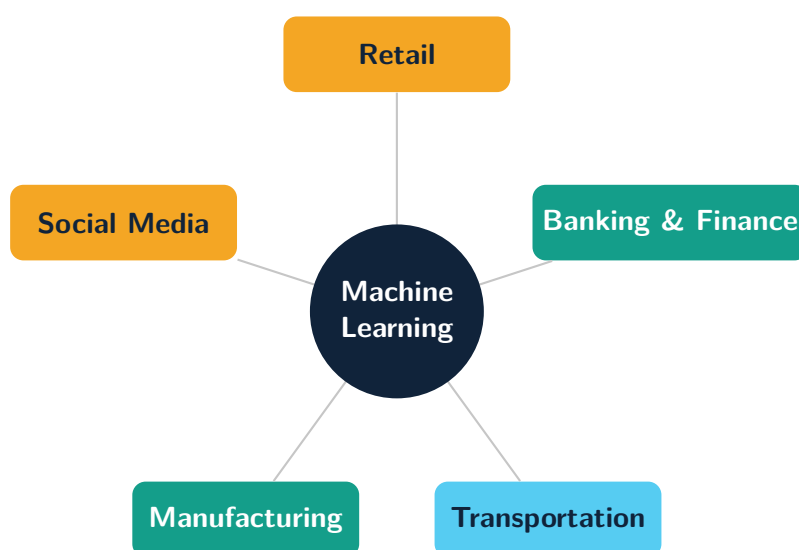


Figure 4.1: Five major B2B application areas of machine learning.

4.2 Retail

4.2.1 Stock Forecasting

Large e-commerce sites like Amazon sell tens of millions of products. During major sales events (e.g., the Great Indian Festival), increasing stock for every single product would be financially unwise and risky. Data analysts and scientists use ML algorithms to analyze past years' sales data and forecast which products are likely to sell well; stock is increased accordingly.

Example — Big-Sale Forecasting

Even a small mistake in a sales forecast can cause losses worth crores of rupees. Companies like Myntra and Flipkart rely on ML-driven forecasting before every big sale.

4.2.2 Customer Profiling and Targeted Marketing

Retail stores (e.g., Big Bazaar, Spencer's) often ask for your phone number at checkout — mainly to track buying behaviour. For instance, someone who frequently buys health-related or milk products may be profiled as “health conscious.” This profiled data (including phone numbers) is often sold to advertisers. Rather than randomly texting a million people, a company can buy targeted data to reach 100 health-conscious customers, resulting in a far higher conversion rate.

“If you are not paying for the product, then you are the product.”

4.2.3 Product Placement (Association Rule Mining)

Deciding which products should be placed next to each other on a shelf is also guided by ML, using a technique called **Association Rule Mining**, which finds correlations between products. If the correlation between two products is strong (e.g., diapers and beer), they are placed close together — the same technique behind the milk-and-eggs market basket analysis of Volume I.

4.3 Banking and Finance

4.3.1 Loan Sanctioning

When you apply for a loan, your profile is first analyzed by an ML algorithm that compares it against the profiles of past loan defaulters. If there is a strong match — e.g., a 63% chance that you might default — the algorithm raises a warning and the bank may reject the application. If the match is low, a loan officer proceeds with manual processing.

ML is also used to decide where to open new branches, what kind of promotional plans to run for customers, and for trading in stock markets.

4.4 Transportation and Logistics

4.4.1 Surge Pricing

Services like Ola and Uber increase fares — sometimes by a factor of 2–3 (e.g., “3.2x”) — during certain periods, such as office commute hours. This happens because demand in an area is high while the supply of cabs is low. To maintain supply, the company entices nearby drivers with the promise of extra earnings, and this extra amount is charged to the passenger. The entire demand-and-supply forecasting process is powered by ML — for example, forecasting cab demand in Kolkata during Durga Puja and repositioning cabs accordingly.

4.4.2 Logistics and Route Optimization

ML is used heavily in delivery and logistics. If a delivery worker has 30 parcels, ML determines the most optimized route for delivering them. Google Maps likewise relies extensively on ML.

4.5 Manufacturing

◇ Definition — Predictive Maintenance

Predictive maintenance uses sensor data and ML models to detect developing faults *before* equipment fails, so that maintenance happens in advance rather than after a breakdown. It is transforming the entire manufacturing sector.

Case Study — Tesla

Tesla takes advance bookings (like iPhone pre-orders) and delivers cars months later, so any delay in production — e.g., a robotic arm malfunctioning — can damage the company's reputation. Tesla therefore uses IoT-based sensors on its robotic arms to continuously monitor metrics such as temperature, RPM, and pressure. If a fault starts developing gradually (e.g., RPM slowly dropping from 300 to 299 to 298), the ML algorithm detects it immediately and alerts engineers — allowing maintenance to happen before the machine actually breaks down.

4.6 Social Media: The Twitter Example

4.6.1 Sentiment Analysis

Around 2012, while Google and Facebook were profitable, Twitter was struggling financially — worrying its investors. Twitter then planned to monetize its data using **sentiment analysis**: a natural language processing (NLP) technique that reads text (e.g., a movie review) and determines whether the sentiment is positive or negative.

4.6.2 Turning Data Into Profit

During elections, when people tweet using a specific hashtag (e.g., #ElectionHashtag), Twitter collects these tweets and uses ML to analyze whether the overall sentiment favors a particular candidate. This effectively creates a real opinion poll — people generally tweet what they believe.

Rather than selling this data to media houses, Twitter sells it to stock brokers and trading firms. If the data suggests a particular political party has a 70% chance of winning, trading firms buy shares of companies aligned with that party (e.g., Reliance) at a low price. Once the election results are declared and share prices rise (e.g., from Rs. 20 to Rs. 120), the shares are sold at a high price for significant profit — a portion of which flows back to Twitter as payment for the data intelligence. The same strategy applies beyond politics — in sports and entertainment as well.

★ Key Takeaways

- ✓ ML is already embedded in everyday products; its B2B impact often outweighs its B2C impact.
- ✓ Retail: demand forecasting, customer profiling with targeted marketing, and associa-

tion rule mining for product placement.

- ✓ Finance: default-risk scoring gates loan decisions before any human review.
- ✓ Transportation: surge pricing and parcel routing are, fundamentally, ML demand/supply and optimization problems.
- ✓ Manufacturing: predictive maintenance converts live sensor streams into advance fault warnings.
- ✓ Social media: sentiment analysis turns raw text into market intelligence — a complete data-to-profit pipeline.

End of Volume II